

ETL COMPLETO DE CIENCIA DE DATOS

ECOMMERCE

Integrantes:

sección: 001D

Juan Pablo Villegas Huichacura

Patrick Gonzáles

Índice

1. Introducción y Objetivos.....	3
1.1 Contexto del Caso.....	3
1.2 Objetivos del Proyecto.....	3
2. Arquitectura de Solución.....	5
2.1 Diagrama de Arquitectura.....	5
2.2 Justificación de Tecnologías.....	5
2.3 Estructura del Proyecto.....	6
3. Pipeline e Integración de Fuentes.....	8
3.1 Detalle de las Fuentes de Datos.....	8
3.2 Transformaciones Robustas.....	9
3.3 Control de Excepciones.....	10
4. API RESTful y Almacenamiento.....	11
4.1 Diseño de la API.....	11
4.2 Seguridad y Configuración.....	12
5. Estrategia de Testing Automatizado.....	13
5.1 Pruebas Unitarias y de Integración.....	13
5.2 Reportes de Cobertura.....	14
6. Contenerización y Despliegue (Docker).....	15
6.1 Configuración de Contenedores.....	15
6.2 Guía de Despliegue.....	16
7. Modelos de Inteligencia Artificial.....	17
7.1 Modelo Supervisado: Predicción de Tiempos de Despacho.....	17
7.2 Modelo No Supervisado: Segmentación de Clientes.....	18
8. Dashboard Interactivo y Visualizaciones.....	20
9. Historial de Cambios (Git Log).....	22
10. Conclusiones y Trabajo Futuro.....	23
10.1 Conclusiones.....	23
10.2 Trabajo Futuro.....	23
11. Bibliografía.....	25

1. Introducción y Objetivos

1.1 Contexto del Caso

El presente proyecto fue desarrollado en el marco del caso de negocio 'Análisis de Mercado E-commerce', propuesto para la asignatura Programación para la Ciencia de Datos (SCY1101) de Duoc UC. El caso plantea la problemática real de una empresa de retail digital con operaciones activas en Chile, Perú y Colombia, que enfrenta un grave problema de fragmentación tecnológica que le impide tener una visión unificada de sus operaciones comerciales y logísticas.

Concretamente, la compañía opera con tres fuentes de datos completamente aisladas entre sí: los datos de transacciones web se procesan de forma independiente en cada país, el registro de comportamiento de navegación de los usuarios (clics y carritos abandonados) se almacena en plataformas de analítica web desconectadas, y los tiempos de despacho junto con los estados de stock en bodega se gestionan en planillas locales y sistemas ERP heterogéneos. Esta desintegración provoca ineficiencias críticas: quiebres de stock no detectados a tiempo, retrasos sistemáticos en las entregas y la imposibilidad de implementar estrategias de precios dinámicos ajustadas a cada mercado regional.

Dado el alto nivel de competitividad del comercio electrónico y la necesidad de fidelizar usuarios mediante experiencias personalizadas, la empresa requiere construir una plataforma analítica unificada capaz de consolidar todas sus fuentes de datos en un pipeline de datos integrado y automatizado.

1.2 Objetivos del Proyecto

El proyecto tiene como objetivo general diseñar, implementar y desplegar una plataforma analítica end-to-end que resuelva el problema de fragmentación de datos descrito. Los objetivos específicos son los siguientes:

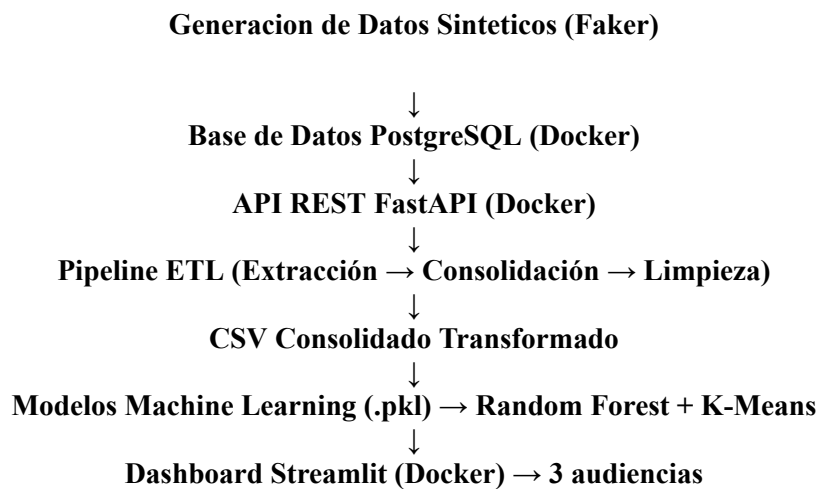
- Implementar un modelo de datos relacional normalizado en PostgreSQL que integre las cuatro entidades principales del negocio: clientes, productos, bodegas y transacciones de venta.

- Desarrollar una API RESTful con FastAPI que exponga los datos almacenados para su consumo por componentes del pipeline.
- Construir un pipeline ETL automatizado que extraiga datos desde la API, los consolide y aplique transformaciones robustas de limpieza, estandarización y enriquecimiento.
- Entrenar e integrar modelos de Machine Learning: un modelo supervisado de predicción de tiempos de despacho (Random Forest Regressor) y un modelo no supervisado de segmentación de clientes (K-Means).
- Desarrollar un dashboard interactivo en Streamlit con visualizaciones diferenciadas para tres audiencias: Gerencia Comercial, Marketing y Operaciones de Bodega.
- Containerizar la solución completa utilizando Docker y docker-compose, asegurando su reproducibilidad y despliegue en cualquier entorno.
- Gestionar el desarrollo del proyecto con metodología Git Flow, manteniendo un historial de cambios limpio y colaborativo.

2. Arquitectura de Solución

2.1 Diagrama de Arquitectura

La arquitectura de la plataforma sigue un flujo de datos lineal y desacoplado por capas, que va desde la generación de datos sintéticos hasta la visualización en el dashboard interactivo. El flujo general implementado es el siguiente:



2.2 Justificación de Tecnologías

A continuación se presenta la justificación de cada librería y herramienta implementada en el proyecto, detallando su versión y propósito específico dentro de la arquitectura.

Libreria / Herramienta	Version	Proposito en el Proyecto
FastAPI	0.137.1	Framework principal para el desarrollo de la API REST. Permite la creación rápida de endpoints con validación automática y documentación Swagger integrada.
Uvicorn	0.49.0	Servidor ASGI de alto rendimiento utilizado para ejecutar la aplicación FastAPI tanto en desarrollo como en el contenedor Docker.

SQLAlchemy	2.0.51	ORM (Object Relational Mapper) utilizado para gestionar la conexión con PostgreSQL y mapear las tablas relacionales a objetos Python en la capa de la API.
psycopg2-binary	2.9.12	Driver de conexión nativa entre Python y PostgreSQL. Actúa como adaptador de base de datos requerido por SQLAlchemy.
Pandas	3.0.3	Librería principal para la manipulación y transformación de datos tabulares en el pipeline ETL. Utilizada para realizar merges, limpieza de nulos, tipado y exportación a CSV.
NumPy	2.4.6	Soporte de operaciones numéricas de alto rendimiento, utilizado como dependencia subyacente de Pandas y Scikit-learn.
Requests	2.34.2	Librería HTTP utilizada en el pipeline ETL para consumir los endpoints de la API REST y obtener los datos en formato JSON.
python-dotenv	1.2.2	Gestión de variables de entorno desde el archivo .env. Garantiza que las credenciales de la base de datos nunca queden expuestas en el código fuente.
Faker	40.23.0	Generación de datos sintéticos realistas. Utilizada para poblar las cuatro tablas de la base de datos con registros coherentes que simulan un escenario real de e-commerce regional.
Scikit-learn	1.9.0	Framework de Machine Learning utilizado para implementar el modelo Random Forest Regressor (aprendizaje supervisado) y el algoritmo K-Means (aprendizaje no supervisado), incluyendo preprocesamiento con StandardScaler y LabelEncoder.
Joblib	1.5.3	Serialización y persistencia de los modelos entrenados en archivos .pkl. Permite cargar los modelos en el dashboard sin necesidad de reentrenarlos.
Streamlit	1.58.0	Framework para el desarrollo rápido del dashboard interactivo. Permite crear interfaces de visualización multi-pestaña con gráficos interactivos y simuladores de predicción en tiempo real.
Plotly	6.8.0	Librería de visualización interactiva integrada con Streamlit. Utilizada para todos los gráficos del dashboard: barras, scatter plots, torta, boxplot.
Pytest	9.1.1	Framework de testing automatizado. Utilizado para implementar las pruebas unitarias del pipeline ETL y las pruebas de integración de la API REST.
HTTPX	0.28.1	Cliente HTTP asincrónico requerido por el TestClient de FastAPI para ejecutar pruebas de integración sobre los endpoints sin necesidad de levantar el servidor manualmente.
Docker / docker-compose	—	Contenerización y orquestación de los tres servicios del proyecto: base de datos PostgreSQL, API REST y dashboard Streamlit.

2.3 Estructura del Proyecto

El proyecto sigue la estructura de directorios recomendada por la asignatura, organizada de forma modular para separar claramente cada responsabilidad dentro de la arquitectura:

Ruta	Descripción
api/main.py	Endpoints RESTful expuestos (Clientes, Productos, Bodegas, Ventas)
api/database.py	Configuración de la conexión y sesión de SQLAlchemy con PostgreSQL
api/Dockerfile	Contenerización del backend de la aplicación FastAPI
dashboards/app.py	Interfaz interactiva y visualizaciones diferenciadas por audiencia en Streamlit
dashboards/Dockerfile	Contenerización del frontend en Streamlit
docker/docker-compose.yml	Orquestador multiservicio: PostgreSQL, API y Dashboard
docker/.env	Archivo protegido con credenciales de base de datos (excluido de Git)
etl/generar_datos.py	Script de simulación e ingesta inicial de datos sintéticos con Faker
etl/pipeline_etl.py	Orquestador central del flujo automático de extracción desde la API
etl/consolidar_datos.py	Script que unifica las cuatro fuentes de datos mediante JOIN
etl/limpiezaDatos.py	Módulo de tipado, joins, limpieza de nulos y normalización
etl/exploratorio.ipynb	Notebook de Análisis Exploratorio de Datos (EDA)
etl/k_means.ipynb	Notebook de entrenamiento del modelo de segmentación RFM con K-Means
etl/predicciones_despacho.ipynb	Notebook de entrenamiento del modelo Random Forest Regressor
data/schema.sql	Script DDL para la creación de tablas en PostgreSQL con restricciones de integridad
data/*_raw.csv	Datos crudos exportados por tabla antes de ser procesados
data/consolidado_transformado.csv	Dataset final unificado, limpio y con conversión a USD
data/clientes_segmentados.csv	Reporte final con los clusters asignados por K-Means
models/*.pkl	Modelos serializados: Random Forest, K-Means, encoders y scaler
docs/modelo_entidad_relacion.png	Evidencia del diseño del modelo de base de datos relacional

3. Pipeline e Integración de Fuentes

3.1 Detalle de las Fuentes de Datos

El pipeline integra tres fuentes de datos diferenciadas, tal como lo establece la arquitectura end-to-end del proyecto. A continuación se detalla cada una:

Fuente 1: Base de Datos Relacional PostgreSQL

Se implementó una base de datos relacional en PostgreSQL, desplegada dentro de un contenedor Docker. El modelo de datos consta de cuatro tablas normalizadas, diseñadas para asegurar la integridad referencial mediante claves primarias y foráneas. El script de creación (data/schema.sql) define las siguientes entidades:

Tabla	Registros	Descripción
clientes_usuarios	10.000	Almacena ID único, país de residencia (Chile, Perú, Colombia), fecha de registro, género y email anonimizado.
catalogo_productos	5.000	Registra nombre, categoría (Electrónica, Vestuario, Hogar), marca y precio base en USD.
operaciones_bodegas	2.000	Detalla ubicación país/ciudad, capacidad de almacenamiento y operador logístico de última milla.
monitoreo_ventas_logistica	50.000	Tabla transaccional principal. Registra fecha/hora, IDs de usuario, producto y bodega, cantidad, precio local, canal de venta (Web/App), clics previos y días de demora real.

Los datos fueron generados mediante el script `etl/generar_datos.py`, utilizando la librería Faker para producir información sintética realista. Las relaciones entre tablas fueron garantizadas durante la generación, asegurando coherencia referencial: los IDs de usuario, producto y bodega presentes en la tabla transaccional existen en sus respectivas tablas maestras.

Fuente 2: API REST (FastAPI)

La API REST desarrollada con FastAPI actúa como capa de exposición de la base de datos y constituye la fuente de extracción principal del pipeline ETL. Expone cuatro endpoints GET, uno por cada tabla, que retornan la totalidad de los registros en formato JSON. El pipeline ETL (`etl/pipeline_etl.py`) consume estos

endpoints mediante la librería requests y convierte las respuestas JSON en DataFrames de Pandas para su posterior procesamiento.

Fuente 3: Archivos CSV Exportados

Como resultado de la extracción desde la API, se generan cuatro archivos CSV en bruto almacenados en la carpeta /data/. Estos archivos representan el estado crudo de los datos antes de cualquier transformación y son posteriormente procesados por el módulo de consolidación y limpieza.

3.2 Transformaciones Robustas

Se implementó un módulo de transformación modular (etl/limpiezaDatos.py) que consolida los datos transaccionales y aplica las siguientes reglas de negocio y limpieza:

- **Imputación de Nulos:** Se identificaron valores nulos en la columna género de la tabla de clientes y se reemplazaron con el valor por defecto 'No Especificado', preservando la totalidad de los registros sin pérdida de información.
- **Estandarización de Texto:** Las columnas categóricas (país, categoría) fueron convertidas a mayúsculas y se eliminaron tildes y caracteres especiales para garantizar homogeneidad en los datos y compatibilidad con los modelos de Machine Learning.
- **Normalización de Fechas:** La columna fecha_hora fue unificada al formato estándar ISO 8601. Adicionalmente, se aplicó una regla estricta de negocio que elimina cualquier registro que contenga fechas en el futuro, consideradas anomalías de datos.
- **Filtrado por Reglas de Negocio:** Se eliminaron transacciones cuyos países de origen no correspondan a Chile, Perú o Colombia, descartando registros inconsistentes con el alcance geográfico del negocio.
- **Conversión de Moneda:** Los precios locales (CLP, PEN, COP) fueron convertidos a una moneda unificada en USD mediante un diccionario dinámico de tasas de cambio, permitiendo comparaciones monetarias estandarizadas entre países.
- **Validación de Integridad:** Se validó que no existieran precios ni cantidades con valores negativos, garantizando la coherencia lógica del dataset.

- Consolidación mediante JOIN: Los cuatro CSV fueron unificados en un único DataFrame mediante operaciones de merge (JOIN) en Pandas, utilizando las claves foráneas correspondientes. Se tuvo especial cuidado en identificar y eliminar columnas duplicadas generadas por el proceso de unión.

El resultado final de este proceso es el archivo `data/consolidado_transformado.csv`, que constituye el dataset limpio y normalizado utilizado como entrada para los modelos de Machine Learning y el dashboard interactivo.

3.3 Control de Excepciones

Se implementó un sistema de control de excepciones profesional utilizando el módulo estandar logging de Python en combinación con bloques try-except estructurados. Las características del sistema son las siguientes:

- Configuración de Logging: Se configuró el módulo logging con nivel INFO y un formato estándar que incluye fecha, hora, nivel de severidad y mensaje descriptivo. Los mensajes de ejecución se emiten por consola (stdout), permitiendo su visualización en tiempo real durante la ejecución del pipeline.
- Bloques Try-Except Estructurales: Cada función principal del módulo de transformación fue envuelta en un bloque try-except independiente. Ante un error crítico (archivo fuente faltante, error de tipo al convertir fechas, fallo de conexión a la API), el sistema captura la excepción específica, registra el error exacto con su trazabilidad completa y detiene la ejecución de forma controlada, evitando colapsos abruptos del pipeline.
- Manejo de Errores HTTP en el ETL: La función `extraer_datos()` del pipeline utiliza el método `raise_for_status()` de la librería `requests`, que lanza una excepción `HTTPError` automáticamente si el servidor retorna un código de estado distinto de 200, garantizando que los errores de red o de disponibilidad de la API sean detectados y manejados de forma explícita.

4. API RESTful y Almacenamiento

4.1 Diseño de la API

La API REST fue desarrollada con el framework FastAPI (version 0.137.1) y expone cuatro endpoints de consulta, uno por cada tabla de la base de datos. La conexión con PostgreSQL se gestiona a través de SQLAlchemy como ORM, configurado en el módulo `api/database.py`.

Los endpoints implementados son los siguientes:

Método	Endpoint	Descripción
GET	/clientes	Retorna la totalidad de los registros de la tabla <code>clientes_usuarios</code> en formato JSON.
GET	/productos	Retorna la totalidad de los registros de la tabla <code>catalogo_productos</code> en formato JSON.
GET	/bodegas	Retorna la totalidad de los registros de la tabla <code>operaciones_bodegas</code> en formato JSON.
GET	/ventas	Retorna la totalidad de los registros de la tabla <code>monitoreo_ventas_logistica</code> en formato JSON.

Método	Endpoint	Descripción
GET(id)	/clientes	Retorna uno de los registros de la tabla <code>clientes_usuarios</code> en formato JSON.
GET(id)	/productos	Retorna uno de los registros de los registros de la tabla <code>catalogo_productos</code> en formato JSON.
GET(id)	/bodegas	Retorna uno de los registros de los registros de la tabla <code>operaciones_bodegas</code> en formato JSON.
GET(id)	/ventas	Retorna uno de los registros de los registros de la tabla <code>monitoreo_ventas_logistica</code> en formato JSON.

La documentación interactiva de la API es generada automáticamente por FastAPI y se encuentra disponible en la ruta `http://localhost:8000/docs` (Swagger UI) una vez que el contenedor Docker del backend se encuentra en ejecución. Esta interfaz permite explorar y probar cada endpoint directamente desde el navegador sin necesidad de herramientas adicionales.

Cada respuesta de la API es serializada automáticamente en formato JSON, con manejo de errores HTTP integrado. En caso de que un recurso no sea encontrado, la API retorna el código de estado HTTP 404 con un mensaje descriptivo.

4.2 Seguridad y Configuración

La seguridad de las credenciales de acceso a la base de datos se gestiona mediante variables de entorno, siguiendo las buenas prácticas de desarrollo profesional. El archivo `.env`, ubicado en la carpeta `/docker/`, contiene las credenciales reales y está protegido mediante su inclusión en el archivo `.gitignore`, impidiendo su exposición en el repositorio remoto.

Se proporciona un archivo `.env.example` en la raíz del proyecto como plantilla de configuración para nuevos colaboradores. Este archivo documenta las variables de entorno requeridas sin exponer valores reales. Las variables configuradas son las siguientes:

- `POSTGRES_USER`: nombre de usuario de la base de datos PostgreSQL.
- `POSTGRES_PASSWORD`: contraseña de acceso a la base de datos.
- `POSTGRES_DB`: nombre de la base de datos del proyecto.
- `POSTGRES_HOST`: nombre del host interno de Docker (`ecommerce_postgres`).
- `POSTGRES_PORT`: puerto de conexión a PostgreSQL (5432).

La librería `python-dotenv` carga automáticamente estas variables al iniciar la aplicación, garantizando que ningún valor sensible quede hardcodeado en el código fuente.

5. Estrategia de Testing Automatizado

5.1 Pruebas Unitarias y de Integración

Se implementó una estrategia de testing automatizado utilizando el framework pytest (version 9.1.1) en combinación con httpx (version 0.28.1). Los tests se encuentran en la carpeta /tests/ y se organizan en dos archivos independientes según su tipo.

Pruebas de Integración de la API: test_main.py

Se implementaron 10 pruebas funcionales utilizando el TestClient de FastAPI, que simula peticiones HTTP reales a la API sin necesidad de levantar el servidor manualmente. Para la ejecución de estas pruebas es necesario que el contenedor Docker con PostgreSQL se encuentre activo, ya que representan pruebas de integración que verifican la comunicación real entre la API y la base de datos. Los tests implementados cubren los siguientes escenarios:

- Endpoint raíz (/): verificación de respuesta HTTP 200 y existencia del mensaje de bienvenida.
- GET /clientes: verificación de HTTP 200, que la respuesta sea una lista y que contenga registros.
- GET /productos: verificación de HTTP 200, respuesta tipo lista con registros existentes.
- GET /bodegas: verificación de HTTP 200, respuesta tipo lista con registros existentes.
- Casos exitosos por ID (clientes, productos, bodegas, ventas): verificación de HTTP 200 y validación del identificador retornado.
- Casos de error por ID inexistente (clientes, productos, bodegas, ventas): verificación de respuesta HTTP 404.

Pruebas Unitarias del Pipeline ETL: test_pipeline.py

Se implementaron pruebas unitarias utilizando pytest en combinación con el módulo unittest.mock, siguiendo el patrón de Mocking para aislar la lógica del pipeline ETL de sus dependencias externas (API y base de datos). Estas pruebas pueden ejecutarse sin necesidad de que Docker, PostgreSQL ni FastAPI se encuentren activos. Los tests implementados son:

- Transformación de datos: se simuló datos JSON equivalentes a la respuesta de la API y se verificó que la función `transformar_datos()` genere un objeto `DataFrame`, mantenga la cantidad correcta de registros y cree las columnas esperadas.
- Manejo de datos vacíos: se entregó una lista vacía como entrada y se verificó que la función retorne un `DataFrame` vacío sin provocar excepciones.
- Manejo de errores HTTP mediante Mocking: se utilizó el decorador `@patch('etl.pipeline_etl.requests.get')` para simular una falla de la API, configurando un `HTTPError` artificial mediante `raise_for_status`. Se verificó que la función `extraer_datos()` propague correctamente la excepción utilizando `pytest.raises`.

Para ejecutar la totalidad de los tests, con Docker activo, se utiliza el siguiente comando:

```
python -m pytest -v
```

El resultado obtenido fue de 15 pruebas aprobadas sobre 15 ejecutadas (15/15 PASSED), validando el correcto funcionamiento de todos los componentes testeados.

5.2 Reportes de Cobertura

Durante el desarrollo del proyecto se validó el comportamiento de los componentes mediante la ejecución exitosa de la suite completa de pruebas, obteniendo un resultado de 15/15 pruebas aprobadas. Si bien no se generó un reporte formal de cobertura utilizando `pytest-cov`, como mejora futura podría incorporarse ejecutando:

```
python -m pytest --cov=api --cov=etl --cov-report=term
```

Esto permitiría obtener el porcentaje exacto de líneas de código cubiertas por las pruebas, proporcionando una métrica formal de calidad del testing implementado.

6. Contenerización y Despliegue (Docker)

6.1 Configuración de Contenedores

La plataforma completa fue contenerizada utilizando Docker y orquestada mediante un archivo `docker-compose.yml` ubicado en la carpeta `/docker/`. La arquitectura de contenedores consta de tres servicios principales y un volumen persistente:

Contenedor 1: postgres

- Imagen: `postgres:16`
- Servicio: motor de base de datos PostgreSQL con las cuatro tablas del modelo de datos.
- Persistencia: volumen nombrado `postgres_data` montado en `/var/lib/postgresql/data`.
- Puerto expuesto: `5432:5432`
- Configuración: credenciales gestionadas mediante variables de entorno desde el archivo `.env`.

Contenedor 2: api

- Construcción: `api/Dockerfile`
- Servicio: API REST FastAPI ejecutada con Uvicorn dentro del contenedor.
- Conexión: se comunica con el contenedor postgres mediante la red interna de Docker, usando el host `ecommerce_postgres`.
- Puerto expuesto: `8000:8000`
- Documentación disponible en: <http://localhost:8000/docs> (Swagger UI)

Contenedor 3: dashboard

- Construcción: `dashboards/Dockerfile`
- Servicio: plataforma analítica interactiva desarrollada en Streamlit.
- Volúmenes montados: `../data` → `/app/data` y `../models` → `/app/models`, para acceder a los datasets transformados y modelos serializados.
- Puerto expuesto: `8501:8501`

- Acceso disponible en: <http://localhost:8501>

6.2 Guía de Despliegue

A continuación se detallan los pasos necesarios para desplegar la plataforma completa desde cero:

Paso 1: Clonar el repositorio

```
git clone <URL_DEL_REPOSITORIO>
cd proyecto-ecommerce
```

Paso 2: Configurar variables de entorno

Crear el archivo `.env` dentro de la carpeta `/docker/` copiando la plantilla disponible en `.env.example` y completando las credenciales correspondientes.

Paso 3: Levantar la plataforma completa

```
cd docker
docker compose up --build
```

Este comando construye las imágenes Docker de la API y el dashboard, levanta los tres contenedores en el orden correcto y deja la plataforma completamente operativa.

Paso 4: Acceder a los servicios

- API REST y documentacion Swagger: <http://localhost:8000/docs>
- Dashboard interactivo: <http://localhost:8501>

Requisitos previos: Docker Desktop con soporte WSL2 (Windows) y Python 3.12 o superior.

7. Modelos de Inteligencia Artificial

El proyecto incorpora dos modelos de Machine Learning complementarios, desarrollados en Jupyter Notebooks dentro de la carpeta /etl/ y serializados con Joblib para su consumo directo por el dashboard interactivo. Los resultados de ambos modelos son visualizados en tiempo real a través de la plataforma Streamlit.

7.1 Modelo Supervisado: Predicción de Tiempos de Despacho

Se implementó un modelo de aprendizaje supervisado basado en el algoritmo Random Forest Regressor, entrenado para predecir los días de demora esperados en el despacho de un pedido. Este modelo fue seleccionado por su capacidad para manejar relaciones no lineales entre variables y su robustez frente al sobreajuste, especialmente al trabajar con variables categóricas previamente codificadas.

Variables de entrada (features):

- país: país del cliente (categórica, codificada con One-Hot Encoding).
- categoría: categoría del producto (categórica, codificada con One-Hot Encoding).
- id_bodega: identificador de la bodega de despacho (categórica, codificada con Label Encoding para optimizar el uso de memoria).
- volumen_compra_dia: feature de ingeniería calculada agrupando el total de transacciones por fecha y bodega, representando la carga operativa del día.

Variable objetivo:

- dias_demora_real: días reales que tomo el despacho del pedido.

Proceso de entrenamiento:

- División del dataset: 80% entrenamiento / 20% prueba.
- Preprocesamiento: One-Hot Encoding para país y categoría, Label Encoding para id_bodega.

- Serializacion: modelo guardado en models/regresion_despacho_model.pkl, columnas en models/columnas_regresion.pkl y encoder en models/encoder_bodegas.pkl.

Metricas de evaluacion (conjunto de prueba):

Metrica	Valor	Interpretacion
MAE	1.36 días	Error promedio de predicción
RMSE	1.70 días	Penaliza errores grandes
R2	0.2035	Varianza explicada por el modelo

El valor de $R^2 = 0.2035$ es esperable dado que la variable objetivo fue generada mediante reglas probabilísticas y aleatorias durante la fase de datos sintéticos, limitando la varianza explicable real. No obstante, el MAE de 1.36 días representa una precisión operacionalmente útil para estimar promesas de entrega al cliente, objetivo principal del modelo en el contexto del negocio.

7.2 Modelo No Supervisado: Segmentación de Clientes

Se implementó un modelo de aprendizaje no supervisado basado en el algoritmo K-Means para segmentar automáticamente la base de clientes según su comportamiento de compra y navegación. El enfoque utilizado corresponde a un análisis RFM Extendido (Recencia, Frecuencia, Valor Monetario y Tasa de Abandonos).

Variables de entrada (calculadas por usuario):

- Recencia: días transcurridos desde la última compra del usuario.
- Frecuencia: cantidad total de compras realizadas por el usuario.
- Valor monetario: gasto total acumulado del usuario en USD.
- Tasa de abandonos: proporción de sesiones del usuario que no culminaron en compra.

Todas las variables fueron escaladas mediante StandardScaler antes del entrenamiento para garantizar que ninguna dimensión domine el cálculo de distancias del algoritmo.

Proceso de entrenamiento:

- Determinación del número de clusters: se aplicó el método del codo evaluando valores de K entre 2 y 10, determinando K = 4 como el punto óptimo de la curva de inercia.
- Evaluación de calidad: Índice de Silueta = 0.2587, indicando una separación moderada entre segmentos, coherente con el carácter sintético de los datos.
- Serialización: modelo guardado en models/kmeans_model.pkl y scaler en models/scaler_kmeans.pkl.
- Resultados: los clusters asignados por usuario fueron almacenados en data/clientes_segmentados.csv para su consumo por el dashboard.

Segmentos identificados:

Cluster	Nombre	Características Principales
0	Vitrineadores (Alta Fricción)	Alta tasa de abandonos, frecuencia media de compra, bajo valor monetario. Usuarios que exploran múltiples productos antes de comprar.
1	Clientes VIP (Alto Valor)	Mayor frecuencia y gasto acumulado, baja recencia. Segmento mas importante desde la perspectiva comercial.
2	Compradores Decididos	Baja tasa de abandono, compras rápidas y directas, valor monetario intermedio. Comportamiento estable y predecible.
3	Inactivos (Riesgo de Fuga)	Alta recencia (muchos días sin comprar), baja frecuencia y bajo valor monetario. Candidatos a campanas de reactivación.

8. Dashboard Interactivo y Visualizaciones

El dashboard interactivo fue desarrollado con Streamlit (version 1.58.0) y se estructura en tres pestanas principales, cada una orientada a una audiencia especifica de la empresa. Los graficos fueron implementados utilizando la libreria Plotly (version 6.8.0) para garantizar interactividad y calidad visual. El dashboard es accesible en <http://localhost:8501> una vez levantado el contenedor Docker correspondiente.

Pestana 1: Gerencia Comercial

Orientada a ejecutivos y directores que requieren indicadores macro para la toma de decisiones estrategicas a nivel regional.

- Grafico 1 — Ingresos Totales por Pais (barras): muestra el total de ingresos en USD por pais (Chile, Peru, Colombia), permitiendo identificar cual mercado genera mayor valor para la compania.
- Grafico 2 — Ticket Promedio por Categoria y Pais (barras agrupadas): contrasta el valor promedio de compra por categoria de producto (Electronica, Vestuario, Hogar) diferenciado por pais, apoyando decisiones sobre estrategias de precio regional.

Pestana 2: Marketing

Orientada a analistas de campanas y equipos de fidelizacion que requieren comprender el comportamiento de los distintos segmentos de clientes.

- Grafico 3 — Distribucion de Segmentos de Clientes (torta): muestra la proporcion porcentual de cada uno de los cuatro clusters obtenidos por K-Means (Vitrineadores, Clientes VIP, Compradores Decididos, Inactivos), permitiendo dimensionar el tamano relativo de cada segmento.
- Grafico 4 — Recencia vs Valor Monetario por Cluster (scatter plot): visualiza la dispersion de los clientes en el espacio RFM, coloreados por cluster comercial, facilitando la interpretacion de las diferencias comportamentales entre segmentos.

Pestana 3: Operaciones y Bodega

Orientada a gestores logísticos que requieren monitorear el rendimiento operativo y anticipar tiempos de entrega.

- Grafico 5 — Distribucion del Retraso por Bodega (boxplot): muestra la distribucion de dias de demora real agrupada por bodega, permitiendo identificar cuales instalaciones presentan mayor variabilidad o retrasos sistematicos.
- Elemento 6 — Simulador de Prediccion de Despacho (modulo interactivo): permite al usuario ingresar pais, bodega, categoria de producto y volumen de compra del día, y obtiene en tiempo real la estimacion de dias de entrega calculada por el modelo Random Forest. Este simulador consume directamente los archivos .pkl serializados, sin necesidad de reentrenamiento.

9. Historial de Cambios (Git Log)

El proyecto fue gestionado utilizando Git con una metodología basada en Git Flow simplificado, adoptando el siguiente esquema de trabajo colaborativo:

- Rama principal main: rama de producción estable. Ningún cambio de desarrollo fue realizado directamente sobre esta rama.
- Ramas feature: cada funcionalidad o componente nuevo del proyecto fue desarrollado en una rama independiente con nombre descriptivo (feature/nombre-funcionalidad).
- Pull Requests: al completar cada rama feature, se creó un Pull Request hacia main, revisado antes de ser fusionado mediante merge.
- Eliminación de ramas: cada rama feature fue eliminada tras su merge exitoso, manteniendo el repositorio limpio.

Durante el desarrollo del proyecto se crearon y fusionaron 13 ramas feature, con sus respectivos 13 Pull Requests. Este historial refleja una metodología de desarrollo incremental y colaborativo, donde cada entrega parcial fue versionada de forma independiente, permitiendo la trazabilidad completa de los cambios realizados por todos los integrantes del equipo.

El archivo `.gitignore` fue configurado para excluir del repositorio remoto los archivos sensibles (`.env`), entornos virtuales (`.venv/`), caches de Python (`__pycache__`) y archivos temporales del sistema operativo, garantizando un repositorio limpio y seguro.

Las evidencias del historial de commits, ramas y Pull Requests pueden visualizarse directamente en el repositorio remoto de GitHub del proyecto.

10. Conclusiones y Trabajo Futuro

10.1 Conclusiones

El desarrollo de este proyecto represento una experiencia integral de construccion de una solucion de datos profesional end-to-end. A lo largo de las cuatro semanas de trabajo, el equipo logro implementar exitosamente todos los componentes solicitados: modelo de datos relacional, API REST, pipeline ETL automatizado, modelos de Machine Learning y dashboard interactivo diferenciado por audiencia, todo ello containerizado mediante Docker.

Una de las lecciones mas importantes que deja este proyecto es la relevancia del orden y la metodologia en el desarrollo de sistemas de datos complejos. Trabajar con ramas Git independientes por cada componente nuevo no solo facilito la colaboracion entre integrantes, sino que tambien permitio identificar y aislar errores de forma mucho mas eficiente. Cada funcionalidad fue entregada de forma incremental, lo que redujo significativamente el riesgo de regresiones y conflictos de codigo.

Sin duda, el mayor desafio tecnico del proyecto fue la etapa de containerizacion con Docker. Integrar tres servicios distintos (base de datos, API y dashboard) en una arquitectura orquestada con docker-compose, asegurando que cada contenedor se comunicara correctamente con los demas a traves de la red interna de Docker, requirio un proceso de prueba, error y ajuste iterativo. La clave para superar este desafio fue abordar el despliegue de forma gradual: primero el contenedor de la base de datos, luego la API validando la conexion, y finalmente el dashboard. Este enfoque paso a paso permitio detectar y corregir problemas de configuracion antes de que se acumularan, convirtiendo una etapa compleja en un proceso manejable y aprendizaje solido.

A nivel de modelos de Machine Learning, se reconoce como limitacion esperada el bajo valor de R^2 obtenido en el modelo de regresion, derivado directamente del uso de datos sinteticos generados de forma aleatoria. Esta limitacion refuerza la importancia de contar con datos reales del negocio para entrenar modelos con capacidad predictiva genuina.

10.2 Trabajo Futuro

Como oportunidades de mejora e incorporacion de nuevas capacidades, se identifican las siguientes lineas de trabajo futuro:

- Integracion con datos reales: reemplazar los datos sinteticos por datos operacionales reales de la empresa, lo que permitiria mejorar sustancialmente la precision de los modelos predictivos y la relevancia de los insights generados por el dashboard.
- Escalabilidad en la nube: migrar la arquitectura Docker a un servicio de contenedores en la nube (AWS ECS, Google Cloud Run o Azure Container Instances) para garantizar disponibilidad continua y escalabilidad horizontal ante aumentos de demanda.
- Implementacion de reporte de cobertura: incorporar pytest-cov para generar reportes formales de cobertura de codigo, estableciendo un umbral minimo de cobertura como requisito de calidad del pipeline de CI/CD.
- Exposicion de modelos via API REST: desarrollar endpoints especificos en FastAPI para servir las predicciones del modelo Random Forest y los segmentos K-Means, completando la arquitectura de microservicios de la plataforma.
- Automatizacion del pipeline ETL: implementar un orquestador de tareas (Apache Airflow o Prefect) para ejecutar el pipeline ETL de forma programada y automatica, simulando la llegada de datos diarios.
- Mejora del dashboard: agregar filtros dinamicos por rango de fechas, categoria de producto y pais, aumentando la granularidad del analisis disponible para cada audiencia.

11. Bibliografía

FastAPI. (2024). FastAPI Documentation. Tiangolo. <https://fastapi.tiangolo.com>

McKinney, W. (2022). Python for Data Analysis (3ra ed.). O'Reilly Media.

pandas Development Team. (2024). pandas Documentation (v3.0). <https://pandas.pydata.org/docs/>

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. <https://scikit-learn.org>

PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/>

Docker Inc. (2024). Docker Documentation. <https://docs.docker.com>

Streamlit Inc. (2024). Streamlit Documentation (v1.58). <https://docs.streamlit.io>

Plotly Technologies Inc. (2024). Plotly Python Graphing Library. <https://plotly.com/python/>

Pytest Development Team. (2024). pytest Documentation (v9.1). <https://docs.pytest.org>

Python Software Foundation. (2024). Python 3.12 Documentation. <https://docs.python.org/3.12/>

Faker Contributors. (2024). Faker Library Documentation. <https://faker.readthedocs.io>

SQLAlchemy Authors. (2024). SQLAlchemy 2.0 Documentation. <https://docs.sqlalchemy.org/en/20/>